

NANYANG TECHNOLOGICAL UNIVERSITY

Data Structures & Algorithms in Python

Binary Trees – Tutorial Answers

Dr. Owen Noel Newton Fernando

College of Engineering

School of Computer Science and Engineering

Q01

1. You are given a pre-order traversal and an inorder traversal of a binary tree. Draw the binary tree from the two traversal results.

   pre-order: A, B, C, D, E, F, G
   inorder: C, B, E, D, A, F, G

Content Copyright Nanyang Technological University

Q01 - Answer

Let's go through the answer step-by-step

- The first visiting node of the pre-order traversal is the **root**. From the in-order traversal, you can find that C, B, E, D are in the left subtree and F, G are in the right subtree.
- The second visiting node of the pre-order traversal is the root of the left subtree if the left subtree is not empty.
- Otherwise, it will be the root of the right subtree. From the in-order traversal, you can find that C is in the left subtree and E,D are in the right subtree. etc.

Content Copyright Nanyang Technological University

Q01 - Answer

Let's go through the answer step-by-step

Given Traversals

Preorder (Root → Left → Right):
A, B, C, D, E, F, G

Inorder (Left → Root → Right):
C, B, E, D, A, F, G

Content Copyright Nanyang Technological University

Q01 - Answer

Let's go through the answer step-by-step

Step 1: Root

- First node in preorder = A.
- So A is the root.

In inorder sequence:
C, B, E, D | **A** | F, G

Left subtree of A: **C, B, E, D**

Right subtree of A: **F, G**

Content Copyright Nanyang Technological University

Q01 - Answer

Let's go through the answer step-by-step

Step 2: Left Subtree of A

- Next in preorder = **B → root of left subtree**.

In inorder left part (C, B, E, D):
C | **B** | E, D

Left of B: **C**

Right of B: **E, D**

Content Copyright Nanyang Technological University

Q01 - Answer

Let's go through the answer step-by-step

Step 3: Left Subtree of B

- Next in preorder = C.
- Matches inorder left (C) → so C is left child of B.

Content Copyright Nanyang Technological University

Q01 - Answer

Let's go through the answer step-by-step

Step 4: Right Subtree of B

- Next in preorder = D → root of right subtree of B.

In inorder right part of B: E, D → split at D.

- Left of D: E
- Right of D: (empty)

So:

- E = left child of D.
- D = right child of B.

Content Copyright Nanyang Technological University

Q01 - Answer

Let's go through the answer step-by-step

Step 5: Right Subtree of A

- After finishing left subtree (B,C,D,E), next preorder = F.
- So F is right child of A.

In inorder right part (F, G):

- Root = F
- Right = G

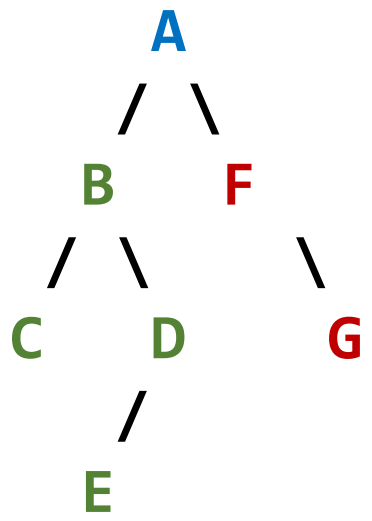
So F has right child G.

Content Copyright Nanyang Technological University

Q01 - Answer

Let's go through the answer step-by-step

Final Tree



Content Copyright Nanyang Technological University

Q02

(levelOrderTraversal) Write an iterative function levelOrderIterative() that prints a level-by-level traversal of the binary search tree using a queue, starting at the root node level. The function uses only enqueue() or dequeue() operations to add or remove nodes from the queue. Ensure the queue is empty at the beginning of the traversal.

The function prototype is given as follows:

```
def levelOrderIterative(root):
```

Content Copyright Nanyang Technological University

Q02 - Answer

Let's go through the answer step-by-step

1. Class Definitions

```
class BSTNode:
    def __init__(self, item):
        self.item = item
        self.left = left
        self.right = right
```

This defines a **Binary Tree Node**.

Each node has:

- **item**: the value stored in the node
- **left**: pointer/reference to the left child node
- **right**: pointer/reference to the right child node

Content Copyright Nanyang Technological University

Q02 - Answer

Let's go through the answer step-by-step

1. Class Definitions

```
class QueueNode:
    def __init__(self, data):
        self.data = data
        self.nextPtr = None
```

class QueueNode:

This defines a class called QueueNode.

Each QueueNode represents one node in a queue implemented as a linked list.

Content Copyright Nanyang Technological University

Q02 - Answer

Let's go through the answer step-by-step

1. Class Definitions

```
class Queue:
    def __init__(self):
        self.head = None
        self.tail = None
```

class Queue:

This defines a Queue data structure.

The queue will be implemented using a linked list of QueueNode objects.
It follows the FIFO (First In, First Out) rule:

- Elements are inserted at the tail (enqueue).
- Elements are removed from the head (dequeue).

Content Copyright Nanyang Technological University

Q02 - Answer

2. Functions

insert_bst_node(node_ref, value)

This function inserts a new value into a Binary Search Tree (BST).

node_ref is a list containing a node ([node]) instead of just a node.

This trick is used because lists are mutable in Python, so we can change the reference (like "pass by reference").

```
def insert_bst_node(node_ref, value):
    if node_ref[0] is None:
        node_ref[0] = BSTNode(value)
    else:
        if value < node_ref[0].item:
            if node_ref[0].left is None:
                node_ref[0].left = BSTNode(value)
            else:
                insert_bst_node([node_ref[0].left], value)
        elif value > node_ref[0].item:
            if node_ref[0].right is None:
                node_ref[0].right = BSTNode(value)
            else:
                insert_bst_node([node_ref[0].right], value)
```

Content Copyright Nanyang Technological University

Q02 - Answer

2. Functions

insert_bst_node(node_ref, value)

If the position is empty (no node), create a new **BSTNode** with **value**.

This becomes:

- the **root** if the tree was **empty**.
- or a **leaf node** if it's a valid spot in the tree.

```
def insert_bst_node(node_ref, value):
    if node_ref[0] is None:
        node_ref[0] = BSTNode(value)
    else:
        if value < node_ref[0].item:
            if node_ref[0].left is None:
                node_ref[0].left = BSTNode(value)
            else:
                insert_bst_node([node_ref[0].left], value)
        elif value > node_ref[0].item:
            if node_ref[0].right is None:
                node_ref[0].right = BSTNode(value)
            else:
                insert_bst_node([node_ref[0].right], value)
```

Content Copyright Nanyang Technological University

Q02 - Answer

2. Functions

insert_bst_node(node_ref, value)

If the spot isn't empty → compare values

If value is smaller, it belongs in the left subtree.

```
def insert_bst_node(node_ref, value):
    if node_ref[0] is None:
        node_ref[0] = BSTNode(value)
    else:
        if value < node_ref[0].item:
            if node_ref[0].left is None:
                node_ref[0].left = BSTNode(value)
            else:
                insert_bst_node([node_ref[0].left], value)
        elif value > node_ref[0].item:
            if node_ref[0].right is None:
                node_ref[0].right = BSTNode(value)
            else:
                insert_bst_node([node_ref[0].right], value)
```

Content Copyright Nanyang Technological University

Q02 - Answer

2. Functions

insert_bst_node(node_ref, value)

If no left child exists → create a new node.

If a left child exists → recursively insert in the left subtree.



```
def insert_bst_node(node_ref, value):
    if node_ref[0] is None:
        node_ref[0] = BSTNode(value)
    else:
        if value < node_ref[0].item:
            if node_ref[0].left is None:
                node_ref[0].left = BSTNode(value)
            else:
                insert_bst_node([node_ref[0].left], value)
        elif value > node_ref[0].item:
            if node_ref[0].right is None:
                node_ref[0].right = BSTNode(value)
            else:
                insert_bst_node([node_ref[0].right], value)
```

Content Copyright Nanyang Technological University

Q02 - Answer

2. Functions

insert_bst_node(node_ref, value)

```
def insert_bst_node(node_ref, value):
    if node_ref[0] is None:
        node_ref[0] = BSTNode(value)
    else:
        if value < node_ref[0].item:
            if node_ref[0].left is None:
                node_ref[0].left = BSTNode(value)
            else:
                insert_bst_node([node_ref[0].left], value)
        elif value > node_ref[0].item:
            if node_ref[0].right is None:
                node_ref[0].right = BSTNode(value)
            else:
                insert_bst_node([node_ref[0].right], value)
```

If value is greater, it belongs in the right subtree.



Content Copyright Nanyang Technological University

Q02 - Answer

2. Functions

insert_bst_node(node_ref, value)

```
def insert_bst_node(node_ref, value):
    if node_ref[0] is None:
        node_ref[0] = BSTNode(value)
    else:
        if value < node_ref[0].item:
            if node_ref[0].left is None:
                node_ref[0].left = BSTNode(value)
            else:
                insert_bst_node([node_ref[0].left], value)
        elif value > node_ref[0].item:
            if node_ref[0].right is None:
                node_ref[0].right = BSTNode(value)
            else:
                insert_bst_node([node_ref[0].right], value)
```

If no right child exists → create new node.

If a right child exists → recurse into the right subtree.

Content Copyright Nanyang Technological University

Q02 - Answer

2. Functions

enqueue(queue, node)

Create a new queue node

Wraps the given node (data) inside a QueueNode object.

A QueueNode stores:

- **data** → the value we want to enqueue
- **nextPtr** → link to the next node in the queue



```
def enqueue(queue, node):
    new_node = QueueNode(node)

    if is_empty(queue.head):
        queue.head = new_node
    else:
        queue.tail.nextPtr = new_node

    queue.tail = new_node
```

Content Copyright Nanyang Technological University

Q02 - Answer

2. Functions

enqueue(queue, node)

Check if the queue is empty

If the queue is empty (**head** is **None**):

- Make the head point to the new node.
- This means the queue now has only one element.



```
def enqueue(queue, node):
    new_node = QueueNode(node)

    if is_empty(queue.head):
        queue.head = new_node
    else:
        queue.tail.nextPtr = new_node

    queue.tail = new_node
```

Content Copyright Nanyang Technological University

Q02 - Answer

2. Functions

enqueue(queue, node)

If the queue is not empty

If the queue already has elements:

- The current last node (tail) should point to the new node.
- This "links" the new node at the end of the queue.



```
def enqueue(queue, node):
    new_node = QueueNode(node)

    if is_empty(queue.head):
        queue.head = new_node
    else:
        queue.tail.nextPtr = new_node

    queue.tail = new_node
```

Content Copyright Nanyang Technological University

Q02 - Answer

2. Functions

enqueue(queue, node)

```
def enqueue(queue, node):
    new_node = QueueNode(node)

    if is_empty(queue.head):
        queue.head = new_node
    else:
        queue.tail.nextPtr = new_node
```

Update the tail pointer

Regardless of whether the queue was empty or not,

- The new node is now the last element.
- So **queue.tail** is updated to point to it.



```
queue.tail = new_node
```

Content Copyright Nanyang Technological University

Q02 - Answer

2. Functions

dequeue(queue)

Check if the queue is empty

If **head** is **None**, the queue is empty.

In that case, we skip to the last line and return None (**nothing to remove**).

```
def dequeue(queue):
    if queue.head is not None:
        node = queue.head.data
        queue.head = queue.head.nextPtr
        if queue.head is None:  # If the queue
becomes empty
            queue.tail = None
        return node
    return None
```

Content Copyright Nanyang Technological University

Q02 - Answer

2. Functions

dequeue(queue)

Store the data of the head node

- The element at the front of the queue (the head) is what we need to dequeue.
- Save its **data** in **node**.



```
def dequeue(queue):
    if queue.head is not None:
        node = queue.head.data
        queue.head = queue.head.nextPtr
        if queue.head is None:  # If the queue
becomes empty
            queue.tail = None
        return node
    return None
```

Content Copyright Nanyang Technological University

Q02 - Answer

2. Functions

dequeue(queue)

Move the head to the next node

- Removes the front node by making **head** point to the next node in line.



```
def dequeue(queue):
    if queue.head is not None:
        node = queue.head.data
        queue.head = queue.head.nextPtr
        if queue.head is None:  # If the queue
becomes empty
            queue.tail = None
        return node
    return None
```

Content Copyright Nanyang Technological University

Q02 - Answer

2. Functions

dequeue(queue)

If the queue becomes empty

After removing the head, if no nodes are left:

- Both **head** and **tail** must be set to **None**.
- This ensures the queue is correctly marked as empty.



```
def dequeue(queue):
    if queue.head is not None:
        node = queue.head.data
        queue.head = queue.head.nextPtr
        if queue.head is None:  # If the queue
becomes empty
            queue.tail = None
        return node
    return None
```

Content Copyright Nanyang Technological University

Q02 - Answer

2. Functions

dequeue(queue)

```
def dequeue(queue):
    if queue.head is not None:
        node = queue.head.data
        queue.head = queue.head.nextPtr
        if queue.head is None:  # If the queue
becomes empty
            queue.tail = None
        return node
    return None
```

Returns the value of the removed node.



Content Copyright Nanyang Technological University

Q02 - Answer

2. Functions

dequeue(queue)

```
def dequeue(queue):
    if queue.head is not None:
        node = queue.head.data
        queue.head = queue.head.nextPtr
        if queue.head is None:  # If the queue
becomes empty
            queue.tail = None
        return node
    return None
```

If no element was there in the first place, return None.



Content Copyright Nanyang Technological University

Q02 - Answer

2. Functions

is_empty(head) and remove_all(node_ref)

Checks whether the queue is empty

A queue is empty if its head pointer is None.

Returns:

True → queue is empty

False → queue has at least one node

```
def is_empty(head):
    return head is None

def remove_all(node_ref):
    if node_ref[0] is not None:
        remove_all([node_ref[0].left])
        remove_all([node_ref[0].right])
        del node_ref[0]
```

Content Copyright Nanyang Technological University

Q02 - Answer

2. Functions

is_empty(head) and remove_all(node_ref)

Base Case

The recursion stops when node_ref[0] is None

if the current node does not exist, there's nothing to delete, so the function returns immediately.



```
def is_empty(head):
    return head is None


def remove_all(node_ref):
    if node_ref[0] is not None:
        remove_all([node_ref[0].left])
        remove_all([node_ref[0].right])
        del node_ref[0]
```

Content Copyright Nanyang Technological University

Q02 - Answer

2. Functions

is_empty(head) and remove_all(node_ref)

```
def is_empty(head):
    return head is None

def remove_all(node_ref):
    if node_ref[0] is not None:
        remove_all([node_ref[0].left])
        remove_all([node_ref[0].right])
        del node_ref[0]
```

Calls **remove_all recursively** on the left child using a new list reference.

Content Copyright Nanyang Technological University

Q02 - Answer

2. Functions

is_empty(head) and remove_all(node_ref)

```
def is_empty(head):
    return head is None

def remove_all(node_ref):
    if node_ref[0] is not None:
        remove_all([node_ref[0].left])
        remove_all([node_ref[0].right])
        del node_ref[0]
```

Calls **remove_all recursively** on the right child.



Content Copyright Nanyang Technological University

Q02 - Answer

2. Functions

is_empty(head) and remove_all(node_ref)

```
def is_empty(head):
    return head is None

def remove_all(node_ref):
    if node_ref[0] is not None:
        remove_all([node_ref[0].left])
        remove_all([node_ref[0].right])
    del node_ref[0]
```

Removes the reference to the current node, effectively deleting it.

Content Copyright Nanyang Technological University

Q02 - Answer

2. Functions

Answer: levelOrderIterative(root)

q is a queue object that will store nodes temporarily to process them level by level.

temp is a temporary pointer used to track the current node.

```
def levelOrderIterative(root):
    q = Queue()
    temp = root

    if temp is not None:
        enqueue(q, temp)
        while not is_empty(q.head):
            temp = dequeue(q)
            print(temp.item, end=' ')
            if temp.left is not None:
                enqueue(q, temp.left)
            if temp.right is not None:
                enqueue(q, temp.right)
```

Content Copyright Nanyang Technological University

Q02 - Answer

2. Functions

Answer: levelOrderIterative(root)

Check if the tree is empty

- If the root is not **None** (i.e., the tree is non-empty), proceed.
- Otherwise, the function does nothing because the tree has no nodes.



```
def levelOrderIterative(root):
    q = Queue()
    temp = root

    if temp is not None:
        enqueue(q, temp)
        while not is_empty(q.head):
            temp = dequeue(q)
            print(temp.item, end=' ')
            if temp.left is not None:
                enqueue(q, temp.left)
            if temp.right is not None:
                enqueue(q, temp.right)
```

Content Copyright Nanyang Technological University

Q02 - Answer

2. Functions

Answer: levelOrderIterative(root)

Enqueue the root node

- The **root** node is added to the **queue**.
- This is the starting point of the **level-order traversal**.



```
def levelOrderIterative(root):
    q = Queue()
    temp = root

    if temp is not None:
        enqueue(q, temp)
        while not is_empty(q.head):
            temp = dequeue(q)
            print(temp.item, end=' ')
            if temp.left is not None:
                enqueue(q, temp.left)
            if temp.right is not None:
                enqueue(q, temp.right)
```

Content Copyright Nanyang Technological University

Q02 - Answer

2. Functions

Answer: levelOrderIterative(root)

Dequeue the next node

- Remove the front node from the queue.
- Print its value.
- This ensures we process nodes in the order they were added, i.e., level by level.

```
def levelOrderIterative(root):
    q = Queue()
    temp = root

    if temp is not None:
        enqueue(q, temp)
        while not is_empty(q.head):
            temp = dequeue(q)
            print(temp.item, end=' ')
            if temp.left is not None:
                enqueue(q, temp.left)
            if temp.right is not None:
                enqueue(q, temp.right)
```

Content Copyright Nanyang Technological University

Q02 - Answer

2. Functions

Answer: levelOrderIterative(root)

```
def levelOrderIterative(root):
    q = Queue()
    temp = root

    if temp is not None:
        enqueue(q, temp)
        while not is_empty(q.head):
            temp = dequeue(q)
            print(temp.item, end=' ')
            if temp.left is not None:
                enqueue(q, temp.left)
            if temp.right is not None:
                enqueue(q, temp.right)
```

Enqueue the children

- Add the **left child first**, then **the right child**, to the queue.
- This ensures the next level is visited in the correct left-to-right order.

Content Copyright Nanyang Technological University

Q02 - Answer

Testing our functions

c is a variable used to store the user's menu choice.

root is a list containing one element (None initially).

```
if __name__ == "__main__":
    c = 1
    root = [None] # Use a list to allow modification of the root reference

    print("1: Insert an integer into the binary search tree;")
    print("2: Print the level-order traversal of the binary search tree;")
    print("0: Quit;")

    while c != 0:
        c = int(input("Please input your choice(1/2/0): "))

        if c == 1:
            i = int(input("Input an integer that you want to insert into the
Binary Search Tree: "))
            insert_bst_node(root, i)

        elif c == 2:
            print("The resulting level-order traversal of the binary search
tree is: ", end="")
            levelOrderIterative(root[0])  # Pass the actual root node
            print()

        elif c == 0:
            remove_all(root)

        else:
            print("Choice unknown;")
```

Content Copyright Nanyang Technological University

Q02 - Answer

Testing our functions

Printing the menu



```
if __name__ == "__main__":
    c = 1
    root = [None]  # Use a list to allow modification of the root reference

    print("1: Insert an integer into the binary search tree;")
    print("2: Print the level-order traversal of the binary search tree;")
    print("0: Quit;")

    while c != 0:
        c = int(input("Please input your choice(1/2/0): "))

        if c == 1:
            i = int(input("Input an integer that you want to insert into the
Binary Search Tree: "))
            insert_bst_node(root, i)

        elif c == 2:
            print("The resulting level-order traversal of the binary search
tree is: ", end="")
            levelOrderIterative(root[0])  # Pass the actual root node
            print()

        elif c == 0:
            remove_all(root)

        else:
            print("Choice unknown;")
```

Content Copyright Nanyang Technological University

Q02 - Answer

Testing our functions

Insert a node (user choice = 1)

Prompts the user to enter an integer.

Calls insert_bst_node with root (the list) and the value.

insert_bst_node will modify the tree in-place, updating root[0] if needed.



```
if __name__ == "__main__":
    c = 1
    root = [None]  # Use a list to allow modification of the root reference

    print("1: Insert an integer into the binary search tree;")
    print("2: Print the level-order traversal of the binary search tree;")
    print("0: Quit;")

    while c != 0:
        c = int(input("Please input your choice(1/2/0): "))

        if c == 1:
            i = int(input("Input an integer that you want to insert into the
Binary Search Tree: "))
            insert_bst_node(root, i)

        elif c == 2:
            print("The resulting level-order traversal of the binary search
tree is: ", end="")
            levelOrderIterative(root[0])  # Pass the actual root node
            print()

        elif c == 0:
            remove_all(root)

        else:
            print("Choice unknown;")
```

Content Copyright Nanyang Technological University

Q02 - Answer

Testing our functions

Level-order traversal (user choice = 2)

Calls levelOrderIterative on the current root (root[0]).

Prints the nodes in level-by-level order.



```
if __name__ == "__main__":
    c = 1
    root = [None]  # Use a list to allow modification of the root reference

    print("1: Insert an integer into the binary search tree;")
    print("2: Print the level-order traversal of the binary search tree;")
    print("0: Quit;")

    while c != 0:
        c = int(input("Please input your choice(1/2/0): "))

        if c == 1:
            i = int(input("Input an integer that you want to insert into the
Binary Search Tree: "))
            insert_bst_node(root, i)

        elif c == 2:
            print("The resulting level-order traversal of the binary search
tree is: ", end="")
            levelOrderIterative(root[0])  # Pass the actual root node
            print()

        elif c == 0:
            remove_all(root)

        else:
            print("Choice unknown;")
```

Content Copyright Nanyang Technological University

Q02 - Answer

Testing our functions

Quit the program (user choice = 0)

If the user chooses 0, the tree is deleted with remove_all.

Exits the loop after this iteration.



```
if __name__ == "__main__":
    c = 1
    root = [None]  # Use a list to allow modification of the root reference

    print("1: Insert an integer into the binary search tree;")
    print("2: Print the level-order traversal of the binary search tree;")
    print("0: Quit;")

    while c != 0:
        c = int(input("Please input your choice(1/2/0): "))

        if c == 1:
            i = int(input("Input an integer that you want to insert into the
Binary Search Tree: "))
            insert_bst_node(root, i)

        elif c == 2:
            print("The resulting level-order traversal of the binary search
tree is: ", end="")
            levelOrderIterative(root[0])  # Pass the actual root node
            print()

        elif c == 0:
            remove_all(root)

        else:
            print("Choice unknown;")
```

Content Copyright Nanyang Technological University

Q03

(**preOrderIterative)** Write an iterative function preOrderIterative() that prints the pre-order traversal of a binary search tree using a stack. The function uses only push() or pop() operations to add or remove nodes from the stack. Ensure the stack is empty at the beginning of the traversal.

The function prototype is given as follows:

```
def preOrderIterative(root):
```

Content Copyright Nanyang Technological University

Q03 - Answer

Let's go through the answer step-by-step

1. Class Definitions

```
class BSTNode:
    def __init__(self, item):
        self.item = item
        self.left = left
        self.right = right
```

This defines a **Binary Tree Node**.

Each node has:

- **item**: the value stored in the node
- **left**: pointer/reference to the left child node
- **right**: pointer/reference to the right child node

Content Copyright Nanyang Technological University

Q03 - Answer

Let's go through the answer step-by-step

1. Class Definitions

```
class StackNode:
    def __init__(self, data):
        self.data = data
        self.next = None
```

StackNode is a node used in a linked-list-based stack implementation.

Each node stores a piece of data and a reference to the next node in the stack.

Content Copyright Nanyang Technological University

Q03 - Answer

Let's go through the answer step-by-step

1. Class Definitions

```
class Stack:
    def __init__(self):
        self.top = None
```

The **Stack** class represents a stack data structure implemented using a linked list.

It allows you to perform standard stack operations such as push, pop, and peek.

Content Copyright Nanyang Technological University

Q03 - Answer

2. Functions

push(stack, node)

Create a new stack node

- Wrap the BST node (**node**) in a **StackNode** object.
- This node will be inserted at the top of the stack.

```
def push(stack, node):
    temp = StackNode(node)

    if stack.top is None:
        stack.top = temp
        temp.next = None
    else:
        temp.next = stack.top
        stack.top = temp
```

Content Copyright Nanyang Technological University

Q03 - Answer

2. Functions

push(stack, node)

Check if the stack is empty

- If the stack is empty (top is None), the new node becomes the top node.
- temp.next = None ensures there's nothing below this node.



```
def push(stack, node):
    temp = StackNode(node)

    if stack.top is None:
        stack.top = temp
        temp.next = None
    else:
        temp.next = stack.top
        stack.top = temp
```

Content Copyright Nanyang Technological University

Q03 - Answer

2. Functions

push(stack, node)

If the stack is not empty

- Set the .**next** of the new node to the current top node.
- Update **stack.top** to point to the new **node**.
- This effectively inserts the new node above the existing stack.



```
def push(stack, node):
    temp = StackNode(node)

    if stack.top is None:
        stack.top = temp
        temp.next = None
    else:
        temp.next = stack.top
        stack.top = temp
```

Content Copyright Nanyang Technological University

Q03 - Answer

2. Functions

pop(s)

Check if the stack is empty

- If the stack is empty, there's nothing to pop, so it returns None.



```
def pop(s):
    if s.top is None:
        return None

    temp = s.top.next
    ptr = s.top.data
    s.top = temp
    return ptr
```

Content Copyright Nanyang Technological University

Q03 - Answer

2. Functions

pop(s)

Store reference to the next node

- temp temporarily holds the node below the top node.
- This is needed to update the stack after removing the top.



```
def pop(s):
    if s.top is None:
        return None

    temp = s.top.next
    ptr = s.top.data
    s.top = temp
    return ptr
```

Content Copyright Nanyang Technological University

Q03 - Answer

2. Functions

pop(s)

Store the data of the top node

- ptr stores the actual value (BST node) that we want to return.



```
def pop(s):
    if s.top is None:
        return None

    temp = s.top.next
    ptr = s.top.data
    s.top = temp
    return ptr
```

Content Copyright Nanyang Technological University

Q03 - Answer

2. Functions

pop(s)

Update the top of the stack

- The new top of the stack becomes the node that was below the original top.
- This effectively removes the original top node from the stack.



```
def pop(s):
    if s.top is None:
        return None

    temp = s.top.next
    ptr = s.top.data
    s.top = temp
    return ptr
```

Content Copyright Nanyang Technological University

Q03 - Answer

2. Functions

pop(s)

Return the popped data

- Returns the BST node that was stored at the top before popping.



```
def pop(s):
    if s.top is None:
        return None

    temp = s.top.next
    ptr = s.top.data
    s.top = temp
    return ptr
```

Content Copyright Nanyang Technological University

Q03 - Answer

2. Functions

peek(s) and isEmpty(s)

peek Function

- Returns the value at the top of the stack without removing it.
- Useful when you want to inspect the top element without modifying the stack.



```
def peek(s):
    if s.top is None:
        return None
    return s.top.data

def isEmpty(s):
    return s.top is None
```

Content Copyright Nanyang Technological University

Q03 - Answer

2. Functions

peek(s) and isEmpty(s)

isEmpty Function

- Checks whether the stack is empty.
- Returns **True** if the stack is empty, **False** otherwise.



```
def peek(s):
    if s.top is None:
        return None
    return s.top.data


def isEmpty(s):
    return s.top is None
```

Content Copyright Nanyang Technological University

Q03 - Answer

2. Functions

Answer: preOrderIterative(root)

Creates an empty stack s.

temp points to the root node of the tree.



```
def preOrderIterative(root):
    s = Stack()
    temp = root

    if temp is None:
        return

    push(s, temp)

    while not isEmpty(s):
        temp = pop(s)
        print(temp.item, end=" ")

        if temp.right is not None:
            push(s, temp.right)
        if temp.left is not None:
            push(s, temp.left)
```

Content Copyright Nanyang Technological University

Q03 - Answer

2. Functions

Answer: preOrderIterative(root)

Base case: If the tree is empty, exit the function.



```
def preOrderIterative(root):
    s = Stack()
    temp = root

    if temp is None:
        return

    push(s, temp)

    while not isEmpty(s):
        temp = pop(s)
        print(temp.item, end=" ")

        if temp.right is not None:
            push(s, temp.right)
        if temp.left is not None:
            push(s, temp.left)
```

Content Copyright Nanyang Technological University

Q03 - Answer

2. Functions

Answer: preOrderIterative(root)

Push the root node onto the stack to start traversal.



```
def preOrderIterative(root):
    s = Stack()
    temp = root

    if temp is None:
        return

    push(s, temp)

    while not isEmpty(s):
        temp = pop(s)
        print(temp.item, end=" ")

        if temp.right is not None:
            push(s, temp.right)
        if temp.left is not None:
            push(s, temp.left)
```

Content Copyright Nanyang Technological University

Q03 - Answer

2. Functions

Answer: preOrderIterative(root)

- Loop while the stack is not **empty**.
- Pop the top node from **the stack** → **temp**.
- Print the value of temp (pre-order visits the node first).



```
def preOrderIterative(root):
    s = Stack()
    temp = root

    if temp is None:
        return

    push(s, temp)

    while not isEmpty(s):
        temp = pop(s)
        print(temp.item, end=" ")

        if temp.right is not None:
            push(s, temp.right)
        if temp.left is not None:
            push(s, temp.left)
```

Content Copyright Nanyang Technological University

Q03 - Answer

2. Functions

Answer: preOrderIterative(root)

Push children

Important: Push right child first, then left child.

- Reason: Stack is LIFO (last-in, first-out).
- Left child will be popped before **right child → preserves pre-order**: **Root → Left → Right**.

```
def preOrderIterative(root):
    s = Stack()
    temp = root

    if temp is None:
        return

    push(s, temp)

    while not isEmpty(s):
        temp = pop(s)
        print(temp.item, end=" ")

        if temp.right is not None:
            push(s, temp.right)
        if temp.left is not None:
            push(s, temp.left)
```

Content Copyright Nanyang Technological University

Q04

(maxDepth) Write a Python function to find the maximum depth of a binary tree.

The function prototype is given as follows:

```
def maxDepth(node):
```

Content Copyright Nanyang Technological University

Q04 - Answer

Now we will go through the maxDepth(node) function

Base Case

- If the **node** is **None**, it means we have reached a **leaf's child** (i.e., an **empty subtree**).
- **Returning -1 means the depth of an empty tree (or edges below a leaf) is -1.**



```
def maxDepth(node):
    if node is None:
        return -1
    else:
        ldepth = maxDepth(node.left)
        rdepth = maxDepth(node.right)

        if ldepth > rdepth:
            return ldepth + 1
        else:
            return rdepth + 1
```

Content Copyright Nanyang Technological University

Q04 - Answer

Now we will go through the maxDepth(node) function

Recursive Calls

- Recursively compute the depth of the **left subtree** (**ldepth**) and **right subtree** (**rdepth**).



```
def maxDepth(node):
    if node is None:
        return -1
    else:
        ldepth = maxDepth(node.left)
        rdepth = maxDepth(node.right)

        if ldepth > rdepth:
            return ldepth + 1
        else:
            return rdepth + 1
```

Content Copyright Nanyang Technological University

Q04 - Answer

Now we will go through the maxDepth(node) function

```
def maxDepth(node):
    if node is None:
        return -1
    else:
        ldepth = maxDepth(node.left)
        rdepth = maxDepth(node.right)
```

Compare Subtree Depths

- Determine which subtree is deeper.
- Add 1 to include the current node in the depth count.
- This ensures the depth returned is the maximum depth from this node to a leaf.



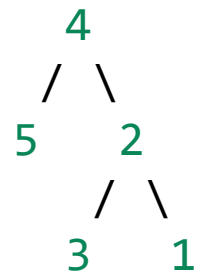
```
if ldepth > rdepth:
    return ldepth + 1
else:
    return rdepth + 1
```

Content Copyright Nanyang Technological University

Q04 - Answer

Now we will go through the maxDepth(node) function

Example



1. Start at root **4**.

2. **ldepth = maxDepth(5)** → **5** is a leaf, so left and right children return **-1**, so **ldepth = max(-1, -1)+1 = 0.**

3. **rdepth = maxDepth(2)** → recursively compute depth for 2's children:

    - **maxDepth(3)** → **leaf** → returns 0
    - **maxDepth(1)** → **leaf** → returns 0
    - So **rdepth = max(0,0)+1 = 1**

4. For root **4**, **max(ldepth, rdepth)+1 = max(0,1)+1 = 2**

The maximum depth of the tree is **2 edges** (or 3 nodes if counting nodes).

Content Copyright Nanyang Technological University